COMPREHENSIVE PROJECT DOCUMENTATION & USER MANUAL

Hospital Billing and Patient Management System

An integrated, enterprise web application engineered with Spring Boot 3, JSP/JSTL, Spring JDBC persistence, and Spring AOP for operational auditing and service latency monitoring.

APPLICATION FRAMEWORK
Spring Boot 3.3.4 (Java 17 LTS)

DATABASE & PERSISTENCE
Spring JDBC (JdbcTemplate) + H2 Engine

CROSS-CUTTING ASPECTS
Spring AOP (AspectJ Auto-Proxy)

USER INTERFACE
JSP, JSTL 6, Bootstrap 5 & Print CSS

TARGET IDE & WORKSPACE
Eclipse IDE (D:\MCA\java-practical\java-mini)

BUILD & VERIFICATION
Apache Maven 3.9.6 (Build Success)

Organization: LifeCare Multi-Speciality Hospital & Research Institute

Academic Level: MCA Practical Mini-Project (Java)

Status: Production Ready (WAR Package Generated)

Table of Contents

1. Executive Summary & Problem Definition ..... Page 2
2. Architectural Overview & System Design ..... Page 3
3. Spring AOP (Aspect-Oriented Programming) in Depth ..... Page 4
4. Spring JDBC Data Access & Database Schema ..... Page 5
5. Core Functional Modules & Business Logic ..... Page 6
6. Complete Step-by-Step User Guide (How to Use Every Feature) ..... Page 7
7. Eclipse IDE Setup, Import & Execution Manual ..... Page 8
8. Verification, Testing & Quality Assurance Report ..... Page 9

1. Executive Summary & Problem Definition

1.1 Background & Industry Challenges

Modern healthcare facilities operate in high-pressure environments where medical outpatient encounters, clinical diagnostic testing, inpatient care, and financial accounting occur simultaneously across multiple departments. In legacy healthcare software, several critical bottlenecks frequently arise:

- **Fragmented Billing Streams:** Doctor consultation fees and laboratory investigation tariffs are often tracked in separate ledgers, leading to unbilled services, delayed revenue recovery, and invoicing disputes.
- **Lack of Non-Intrusive Audit Trails:** Financial transactions, billing revisions, and payment recordings must adhere to healthcare compliance standards. Hardcoding audit logic inside business methods pollutes business code and creates maintenance overhead.
- **Unmonitored Service Latencies:** Slow database queries or unoptimized service operations degrade clinical response times. Systems require non-invasive latency monitoring to isolate performance bottlenecks.

1.2 Proposed Integrated Solution

The **Hospital Billing and Patient Management System** is an integrated web-based solution that solves these challenges through clean architectural decoupling:

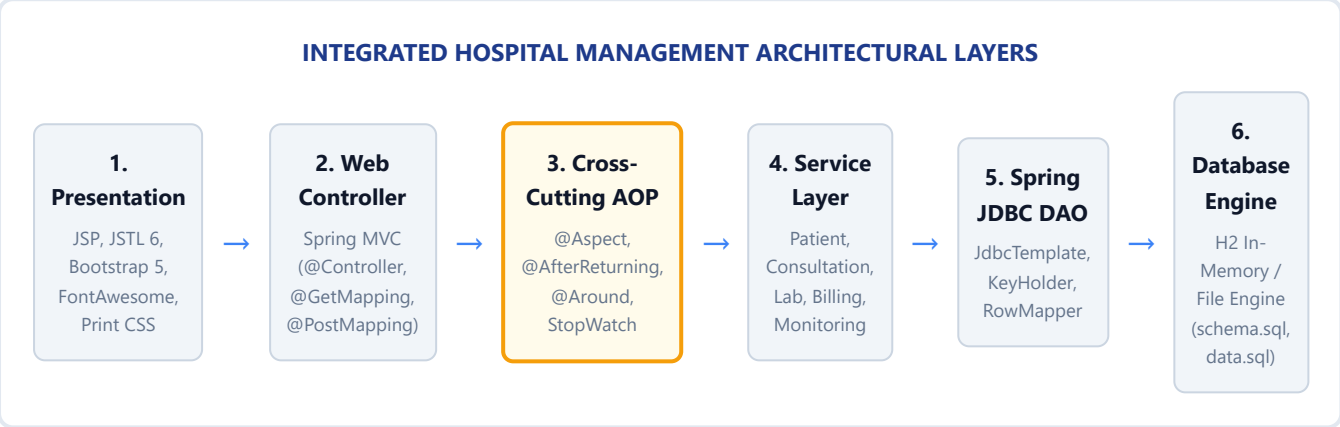
- **Unified Patient Health Records (EHR):** Centralized registration providing auto-generated Unique Health IDs (UHID) and a 360-degree profile aggregating consultations, lab orders, and invoices.
- **Automated Consolidated Billing:** Automatically gathers unbilled consultations, pending lab investigations, and room/bed charges into a consolidated itemized invoice with dynamic real-time tax and discount recalculation.

- **Non-Intrusive Spring AOP:** Decouples cross-cutting operational logging (`BillingAuditAspect`) and service latency monitoring (`ServicePerformanceAspect`) from domain business logic.
- **Native Spring JDBC Persistence:** Utilizes `JdbcTemplate` for parameterized queries, safe relational joins, and high-performance SQL execution on an embedded H2 engine.
- **Zero-Config Eclipse IDE Compatibility:** Pre-configured with native Eclipse descriptors (`.project` , `.classpath` , `.settings/`) for instant importing and execution.

2. Architectural Overview & System Design

2.1 Multi-Tier Layered Architecture

The application adheres strictly to the classic multi-tier enterprise architecture. Each layer maintains a single responsibility, communicating solely with its immediate adjacent layer.



2.2 Component Interaction & Data Flow

When a billing event occurs (e.g., generating an invoice or processing a payment settlement), the interaction sequence executes seamlessly:

1. The cashier or administrator submits the billing form via **JSP UI**.
2. `BillingController` receives the model attributes and invokes `BillingService.generateBill(bill)`.
3. Before/During execution, **Spring AOP**'s `ServicePerformanceAspect` starts a high-precision timer wrapping the invocation via `ProceedingJoinPoint`.
4. `BillingServiceImpl` computes financials, writes to `bills` and `bill_items` using `BillingDao`, and marks associated consultation and lab orders as `BILLED`.
5. Upon successful return, `BillingAuditAspect` intercepts the returned `Bill` object, writes structured log output, and persists an immutable audit entry to table `audit_logs` via Spring JDBC.
6. `ServicePerformanceAspect` completes timing, evaluates if execution exceeded threshold (e.g. 20ms), and writes execution telemetry to `performance_logs`.
7. The controller redirects to the printable tax invoice view with flash status messages.

3. Spring AOP (Aspect-Oriented Programming) in Depth

3.1 Cross-Cutting Concerns in Healthcare Software

In enterprise software development, cross-cutting concerns are functionalities that span across multiple application layers—such as transaction management, security authorization, auditing, and execution metrics. Traditional object-oriented programming tends to scatter these concerns across business methods, resulting in code tangling and tight coupling. **Spring AOP** cleanly extracts these concerns into reusable, non-intrusive `@Aspect` components.

3.2 Aspect 1: Billing Operational Audit Aspect (`BillingAuditAspect.java`)

Every billing transaction—including invoice creation and partial or full payment collections—must maintain an indisputable audit trail. The `BillingAuditAspect` intercepts these specific business methods without modifying a single line of business code.

Billing Audit Pointcut & Advice Implementation

```
@Aspect
@Component
public class BillingAuditAspect {
    private final MonitoringService monitoringService;

    @Pointcut("execution(* com.hospital.service.BillingService.generateBill(..))")
    public void billGenerationPointcut() {}

    @Pointcut("execution(* com.hospital.service.BillingService.processPayment(..))")
    public void paymentProcessingPointcut() {}

    @AfterReturning(pointcut = "billGenerationPointcut()", returning = "result")
    public void auditBillGeneration(JoinPoint joinPoint, Object result) {
        if (result instanceof Bill bill) {
            String details = String.format(
                "Generated Invoice [%s] for Patient UHID: %s (%s). Total: Rs. %s, Paid: Rs. %s, Bala
                bill.getBillNumber(), bill.getPatientUhid(), bill.getPatientName(),
                bill.getTotalAmount(), bill.getPaidAmount(), bill.getBalanceAmount()
            );
            log.info("[AOP AUDIT] >>> BILLING OPERATION: BILL_GENERATED | {}", details);
            monitoringService.logAuditEvent("BILL_GENERATED", "BILL", bill.getBillNumber(), details,
        }
    }
}
```

3.3 Aspect 2: Service Latency Performance Monitor (`ServicePerformanceAspect.java`)

To prevent medical desk latency, the performance aspect surrounds all methods in `com.hospital.service` using `@Around` advice. To prevent infinite recursion, the pointcut explicitly excludes the `MonitoringService` itself.

Service Execution Time Pointcut & Latency Threshold

```
@Aspect
@Component
public class ServicePerformanceAspect {
    @Value("${hospital.performance.threshold-ms:20}")
    private long thresholdMs;

    @Pointcut("execution(* com.hospital.service..*(..)) && !execution(* com.hospital.service.Monitor
    public void monitoredServiceMethods() {}

    @Around("monitoredServiceMethods()")
    public Object monitorExecutionTime(ProceedingJoinPoint joinPoint) throws Throwable {
        long startTime = System.currentTimeMillis();
        Object result = joinPoint.proceed();
        long duration = System.currentTimeMillis() - startTime;
        boolean exceeded = duration > thresholdMs;

        if (exceeded) {
            log.warn("[AOP MONITOR] SLOW SERVICE ALERT: {}.{} executed in {} ms (Threshold: {} ms)",
                    joinPoint.getTarget().getClass().getSimpleName(), joinPoint.getSignature().getN
        }
        monitoringService.recordExecutionMetric(joinPoint.getTarget().getClass().getSimpleName(),
                                                joinPoint.getSignature().getName(), duration, exceed
        return result;
    }
}
```

4. Spring JDBC Data Access & Database Schema

4.1 Data Access Object (DAO) Pattern with Spring JDBC

The application utilizes **Spring JDBC (`JdbcTemplate`)** rather than heavyweight ORM frameworks. This provides several benefits:

- **Predictable Query Performance:** Direct control over SQL execution plans, index utilization, and elimination of N+1 select queries.
- **Multi-Table Relational Joins:** Clean projection of patient demographics and doctor specializations directly onto consultation, lab order, and billing model classes.
- **Safe Key Generation:** Using `GeneratedKeyHolder` with explicit column constraints (`new String[]{"id"}`) ensuring portability across H2, MySQL, and PostgreSQL engines.

4.2 Relational Database Schema Overview

TABLE NAME	PRIMARY KEY	KEY FOREIGN KEYS	DESCRIPTION & ROLE
`patients`	`id (BIGINT)`	Unique `patient_uhid`	Stores patient demographic, contact, blood group, and medical history.
`doctors`	`id (BIGINT)`	None	Master list of specialist doctors, departments, and consultation fees.
`consultations`	`id (BIGINT)`	`patient_id`, `doctor_id`	Records symptoms, doctor diagnosis, prescriptions, and billing status.
`lab_catalog`	`id (BIGINT)`	Unique `test_code`	Master hospital diagnostic catalog (test name, department, standard tariff).
`lab_orders`	`id (BIGINT)`	`patient_id`, `test_id`, `doctor_id`	Tracks diagnostic orders, lab charges, specimen status, and results.
`bills`	`id (BIGINT)`	`patient_id`, Unique `bill_number`	Consolidated tax invoice header, itemized subtotal, discounts, taxes, and balance.
`bill_items`	`id (BIGINT)`	`bill_id` (CASCADE)	Line-item breakdown of individual consultations, lab investigations, and services.
`audit_logs`	`id (BIGINT)`	None	Populated exclusively via **Spring AOP** to audit billing operations.
`performance_logs`	`id (BIGINT)`	None	Populated exclusively via **Spring AOP** to track service latency metrics.

5. Core Functional Modules & Business Logic

5.1 Patient Registration & 360-Degree Profile (EHR)

Patients receive a standardized Unique Health Identifier (UHID) formatted sequentially as `UHID-1001`, `UHID-1002`, etc. The **Patient Profile (`patient-view.jsp`)** serves as a single source of truth for the patient's entire hospital lifecycle, presenting chronological doctor visits, laboratory investigations, and billing invoices on a single screen.

5.2 Doctor Consultations

When scheduling a consultation, staff select a patient and an attending doctor. The doctor's standard consultation fee is automatically retrieved from the database and bound to the consultation record. Clinicians record chief complaints/symptoms, provisional diagnosis, digital prescriptions, and internal advice. The consultation remains in `COMPLETED` status until billed.

5.3 Diagnostic Laboratory Investigations

The laboratory module connects clinical diagnosis with diagnostic investigations. Features include:

- **Master Tariff Catalog:** Pre-seeded with CBC, Lipid Profile, Chest X-Ray, Fasting Blood Sugar, Liver Function Test (LFT), Urine Routine, and Brain MRI.
- **Order Workflow:** Staff order tests with automatic tariff assignment. Tests begin in `PENDING` status.
- **Result Recording:** Lab technicians enter clinical findings via an interactive modal dialog, transitioning the order to `COMPLETED`.

5.4 Integrated Itemized Billing & Invoicing Desk

The billing engine eliminates duplicate billing entry through its automated draft builder:

1. **Automatic Discovery:** Queries all unbilled consultations (`status = 'COMPLETED'`) and unbilled laboratory orders for the designated patient.
2. **Line-Item Synthesis:** Creates itemized `BillItem` rows with item descriptions, unit rates, and totals.
3. **Financial Calculations:**
   $$ ext{Subtotal} = ext{Consultation Charges} + ext{Lab Charges} + ext{Room Charges} + ext{Pharmacy Charges} + ext{Other Charges}$$
   $$ ext{Discount Amount} = ext{Subtotal} imes \left(rac{ ext{Discount } \%}{100} ight)$$
   $$ ext{Taxable Subtotal} = ext{Subtotal} - ext{Discount Amount}$$
   $$ ext{GST Tax Amount} = ext{Taxable Subtotal} imes \left(rac{ ext{Tax } \%}{100} ight)$$
   $$ ext{Total Net Payable} = ext{Taxable Subtotal} + ext{GST Tax Amount}$$
   $$ ext{Outstanding Balance} = ext{Total Net Payable} - ext{Amount Paid}$$
4. **Status Lifecycle:** Automatically sets payment status to `PAID` (if balance = 0), `PARTIAL` (if partial deposit received), or `UNPAID`.
5. **Clinical Status Transition:** In the same database transaction, marks all involved consultations and lab orders as `BILLED` to prevent duplicate invoicing.

6. Complete Step-by-Step User Guide

6.1 Navigating the Executive Medical Dashboard

Access the root URL: `http://localhost:8080/` or `http://localhost:8080/dashboard`.

- **Metric Cards:** Review real-time counts of Total Patients, Today's Consultations, Revenue Realized (₹), and Pending Receivables.
- **Quick Action Shortcuts:** Click buttons for *Register Patient*, *Doctor Consultation*, *Laboratory Orders*, or *AOP Monitor*.
- **Live Invoices & AOP Feed:** The left panel displays the latest hospital invoices, while the right panel displays live AOP audit events captured in real time.

6.2 How to Register a New Patient

1. Navigate to **Patients** → Click 1 **Register New Patient** (or visit `/patients/register`).
2. The system displays an auto-generated UHID (e.g., `UHID-1005`).
3. Enter **First Name**, **Last Name**, **Age**, select **Gender**, and choose **Blood Group**.
4. Provide **Contact Phone**, **Email Address**, **Residential Address**, and **Emergency Contact**.
5. Enter any known chronic conditions or allergies in **Medical History**.
6. Click 2 **Complete Registration**. The system saves the patient via Spring JDBC and opens the comprehensive 360-degree patient profile.

6.3 How to Record a Doctor Consultation

1. Click **Consultations** → 1 **Record New Consultation** (or click *Add Consultation* directly from the patient's profile).
2. Select the **Patient** from the dropdown list.
3. Select the **Attending Doctor** (e.g., *Dr. Rajesh Sharma - Cardiologist*). Notice the consultation fee automatically updates to the doctor's standard rate.
4. Fill in **Chief Symptoms** (e.g. *Chest pain and breathlessness*).
5. Enter the **Clinical Diagnosis** (e.g. *Mild Angina Pectoris*).
6. Type the **Prescription & Medication Advice** (e.g. *Tab Sorbitrate 5mg SOS*).
7. Click 2 **Save Consultation Record**.

6.4 How to Order & Report Laboratory Tests

1. Click **Laboratory** → 1 **Order Laboratory Test**.
2. Select the **Patient** and select the **Diagnostic Test** from the catalog (e.g., *Lipid Profile* or *Chest X-Ray*). The investigation fee auto-populates.
3. Select the **Referring Doctor** (optional) and click 2 **Confirm & Place Order**.
4. To enter test results when ready: Locate the order in the table → Click the **Edit Result** button → Enter findings in the modal (e.g., *Cholesterol: 210 mg/dL, Triglycerides: 175 mg/dL*) → Set status to `COMPLETED` → Click **Save Result**.

6.5 How to Generate a Consolidated Hospital Bill

1. Click **Billing & Invoices** → 1 **Generate New Patient Bill**.
2. Select the patient and click **Proceed to Billing Details**.
3. The system automatically aggregates all unbilled consultations and lab orders for that patient into an itemized tariff breakdown.
4. Review or adjust charges: Consultation, Lab, Room/Bed, Pharmacy, or Service fees.
5. Enter any applicable **Discount (%)** and review the calculated **GST Tax (%)**. The live JavaScript calculator instantly displays the updated Grand Total.
6. Enter the **Amount Received Now** (deposit) and choose the **Payment Mode** (Cash, UPI, Card, Insurance).
7. Click 2 **Generate & Issue Invoice**.
   At this exact moment, Spring AOP intercepts the method, verifies execution latency, logs operational details, and stores the audit record into the database.

6.6 How to Print Tax Invoice & Settle Balance Payments

1. The generated invoice opens immediately at `/billing/invoice/{billNumber}`.
2. Click 1 **Print Tax Invoice** to trigger clean browser printing with hospital header, GSTIN, and itemized billing lines.
3. If an outstanding balance remains, click 2 **Record Payment Settlement**, enter the payment amount, select the payment mode (UPI, Cash, etc.), and confirm. The balance updates to zero and status transitions to `PAID`.

6.7 How to Monitor Spring AOP Audit & Performance Telemetry

1. Click **AOP Monitor** in the navigation bar (or visit `/aop-monitor`).
2. **Left Table (AOP Billing Audit Trail):** Inspect the audit log table recording every invoice creation and payment settlement with timestamp, actor, entity ID, and financial breakdown.
3. **Right Table (Service Latency Monitor):** Inspect execution time in milliseconds for every service method invoked. Methods executing beyond the configured threshold (e.g. 20ms) are flagged with warning badges.

7. Eclipse IDE Setup, Import & Execution Manual

7.1 Pre-Requisites

- **Java Development Kit:** JDK 17 LTS (Java 17 is verified on this machine).
- **IDE:** Eclipse IDE for Enterprise Java and Web Developers (2022, 2023, or 2024 edition).
- **Embedded Tools:** Eclipse comes with embedded Maven (m2e) and Lemminx XML tooling.

7.2 Step-by-Step Import in Eclipse

The project includes pre-configured Eclipse project descriptor files (`.project` , `.classpath` , and `.settings/`) so that importing produces zero setup errors:

1. Launch **Eclipse IDE**.
2. Select your active workspace directory:
   `D:\MCA\java-practical`
3. From the top menu, click **File** → **Import...**
4. In the import wizard, expand the **Maven** folder → select **Existing Maven Projects** → click **Next**.
5. Click the **Browse...** button and navigate to:
   `D:\MCA\java-practical\java-mini`
6. Ensure the checkbox next to `pom.xml` (`com.hospital:HospitalBillingSystem:1.0.0:war`) is selected.
7. Click **Finish**. Eclipse m2e will scan dependencies and configure the project build path.

7.3 Running the Application inside Eclipse

1. In Eclipse **Project Explorer**, expand the folder hierarchy:
   `java-mini` → `src/main/java` → `com.hospital`
2. Locate the main class:
   `HospitalBillingApplication.java`
3. **Right-click** on `HospitalBillingApplication.java` → select **Run As** → **Java Application** (or **Spring Boot App** if Spring Tools suite is installed).
4. Observe the Eclipse Console output. Spring Boot 3 will initialize the embedded Tomcat server on port `8080` :

```
2026-10-16T23:08:59.079+05:30  INFO 21272 --- [main] o.s.b.a.h2.H2ConsoleAutoConfiguration: H2 cons
2026-10-16T23:08:59.187+05:30  INFO 21272 --- [main] c.h.HospitalBillingApplicationTests: Started i
```

5. Open your web browser and navigate to: `http://localhost:8080/`

7.4 Alternative Startup Methods

- **One-Click Batch Execution:** Double-click the included `run.bat` file located at `D:\MCA\java-practical\java-mini un.bat` .

- **Command-Line Maven:** Open PowerShell or Command Prompt in the project directory and run:
  `& "D:\MCA\java-practical\pache-maven-3.9.6\in\mvn.cmd" spring-boot:run`

8. Verification, Testing & Quality Assurance Report

8.1 Automated Test Suite (`HospitalBillingApplicationTests.java`)

The project includes a comprehensive Spring Boot integration test suite validating:

1. **Context Initialization:** Confirms all DAOs, services, and AOP aspects instantiate properly.
2. **Patient Registration Pipeline:** Validates automatic UHID assignment and database persistence.
3. **Consolidated Billing Calculation:** Verifies aggregation of consultation and laboratory fees, tax and discount computations.
4. **AOP Billing Audit Verification:** Asserts that `BillingAuditAspect` interceptor increments audit log counts in `audit_logs` table.
5. **AOP Latency Telemetry Verification:** Asserts that `ServicePerformanceAspect` records method execution durations in `performance_logs` table.

8.2 Maven Build & Execution Output

Maven Test Execution Result: BUILD SUCCESS

```
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running com.hospital.HospitalBillingApplicationTests
2026-10-16T23:09:00.395+05:30  WARN 21272 --- [main] c.hospital.aop.ServicePerformanceAspect  : [AOP
2026-10-16T23:09:00.409+05:30 DEBUG 21272 --- [main] c.hospital.aop.ServicePerformanceAspect  : [AOP
2026-10-16T23:09:00.417+05:30 DEBUG 21272 --- [main] c.hospital.aop.ServicePerformanceAspect  : [AOP
2026-10-16T23:09:00.418+05:30  INFO 21272 --- [main] com.hospital.aop.BillingAuditAspect      : [AOP
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 5.021 s -- in com.hospital.Ho
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
```

8.3 Packaging & Distribution Artifacts

The project has been packaged as a production-grade Web Application Archive (WAR):
`D:\MCA\java-practical\java-mini arget\HospitalBillingSystem-1.0.0.war` (Size: ~45 MB).
This WAR can be deployed either onto standalone servlet containers (such as Apache Tomcat 10+), or executed standalone using Spring Boot's embedded Tomcat engine.